AGILE METHODOLOGY SELECTION REPORT

Project: Fly-by-Wire Arduino System for Emergency Aircraft Fin Control Using a 3D-Printed Aircraft Model

Christian Janse van Rensburg (601840)

Alexander Solomon (601638)

Johni Kuger (601710)

Tshiamo Maise (602178)

Luthando Sibusiso Mbili (602179)

Tumelo Bapela (602088)

PMM 38(7)1 Assignment 2

Table of Contents

Executive Summary

This report evaluates and selects the most appropriate Agile methodology for the development of a scaled fly-by-wire prototype using an Arduino IoT platform. After analysing Scrum, Kanban, and Extreme Programming (XP) against project-specific criteria and the ISO/IEC 25010:2023 (SQuaRE) quality model, Scrum has been selected. Scrum provides the necessary structure to manage the parallel development of hardware, software, and system integration testing while meeting academic deliverables.

1. Project Overview and Context

- Project Description: The project consists of the development of a prototype of a scaled fly-by-wire system that simulates to correct the flight of drones in emergency situations by the controlled movement of rudder, ailerons and elevator surfaces.
- Scope of Work: Working with the Arduino to develop a flight control system to control the surfaces of the aircraft with a servo motor, simulations and testing of the prototype.
- Key Constraints: A challenging task of the project is the integration of hardware (3D printed model, MPU6050, servos) and software (PID control, IoT dashboard).
- Stakeholders: The academic supervisors, project team members and evaluation committees from Belgium Campus iTversity.
- High-Level System Design: A Flight Control Computer (Arduino) receives commands from a sidestick and feedback from various sensors. It uses control laws to send orders to actuators (servos), which move control surfaces to generate an aircraft response.

2. Overview of Agile Principles

Agile development methodologies were established to address the problem of delivering software on time while managing constantly varying requirements. Using Agile methods in embedded systems can be difficult as hardware requirements are usually quite rigid;

nonetheless, an iterative approach can enhance overall efficiency considerably (Wang, et al., 2018). The team can build and test components gradually in Agile, which lessens the risk of horrible integration failure at the end of the project.

3. Selection of Agile Methodologies

Three Agile methodologies are considered for this hardware-software integration project:

1. Scrum: The Scrum framework develops projects by dividing larger tasks into smaller, timeboxed development stages known as Sprints (TOJDEL, 2017). It involves five primary time-based events, including Sprint Planning, Daily Scrums, and Sprint Reviews (BMC Blogs, 2025). Scrum is widely praised for increasing productivity, efficiency, and quality, even when adapted for hardware development processes (Backblaze, 2015)
2. Kanban: Kanban is a visual methodology designed to improve flow and provoke system improvement through visualization and by strictly controlling work in progress (BMC Blogs, 2025). Everything is placed on a Kanban board, showing the entire set of tasks to be completed, pending items, and acquired resources (TOJDEL, 2017).
3. Extreme Programming (XP): XP focuses heavily on software engineering practices like Test-Driven Development (TDD) and continuous integration. Methodologies combining practices of Scrum and XP are often used in embedded systems to promote flexibility and iterative hardware and software partitioning (Cordeiro, et al., 2007).

4. Development of evaluation setup.

To selectively choose the best methodology, the following project-specific criteria were established.

- Hardware/Software Integration Support: The methodology should allow for physical build times as well as software programming.

- Complexity Management: The project involves distinct phases with highly technical requirements. This complexity must be broken down by the methodology.
- Risk Management Capability: The process needs to assist early identification of sensor miscalibration and hardware failure during disturbances simulation through risk management capability.
- Suitability for Academic Delivery: The suitability of academic delivery means the delivery must fit the formal stages needed for the final project report indicating testing results.

5. Quality Criteria (SQuaRE-Based)

1. Reliability

   Reliability refers to the degree to which a system performs its specified functions consistently under defined conditions over a given period (Sheppard, 2025). In context of the fly-by-wire, this means the servo motors controlling the rudder, elevator and ailerons must respond accurately and consistently to every command issued by the Arduino, regardless of how many test cycles have been run or what environmental disturbances are being simulated.

   How each Agile methodology supports Reliability:

   - Scrum: Scrum supports reliability through its sprint structure. As Watt (2014) explains, Scrum is built of principle of iterating in short cycles, demonstrating what was built at the end of each sprint and then testing it before the next cycle begins. This ensures that reliability issues are caught and addressed rather than accumulating until the end of project.
   - Kanban: Kanban supports reliability by controlling the volume of work in progress at any one time. By limiting how many tasks can be active simultaneously, the team reduces the risk of introducing untested changes into a system where hardware and software are tightly coupled.
   - Extreme Programming (XP): XP supports reliability through its engineering discipline. Watt (2014) explains that Agile methodologies such as XP emphasise on improving engineering practices and tools. Practices such as test-driven development and continuous integration mean that every

component is tested before and after integration, which is critical to for an embedded real-time control system.

2. Performance Efficiency

   Performance Efficiency measures how well a system uses its resources relative to the level of performance it delivers under stated conditions (Sheppard, 2025). For the fly-by-wire Arduino system, performance efficiency is a critical quality characteristic because the microcontroller must process the sensor data and generate control outputs within strict timing limits. Any processing delay in reading from the inertial measurement unit (IMU) or in writing PWM values to the servo actuator could cause the aircraft model to respond too slowly to destabilising forces, resulting in a loss of control during a test.

   How each Agile methodology supports Performance Efficiency:

   - Scrum: Scrum supports the performance efficiency by allowing the team to define performance specific acceptance criteria within sprint planning. Watt (2014) explains that Scrum sprint planning involves the team and product owner setting goals and determining what will be worked on. Performance targets, such as maximum sensor polling intervals its actuator response times can be included in these sprint goals and verified before a sprint is closed.
   - Kanban: Kanban supports performance efficiency through its continuous delivery model. When a performance is bottleneck is identified during hardware testing, a fix can be introduced into the Kanban workflow immediately without waiting for a sprint cycle to end, which allows for the team to respond quickly to real-time performance.
   - Extreme Programming (XP): XP supports performance efficiency through its engineering robustness. Watt (2014) identifies improving engineering practices and tools as a core Agile responsibility and XP's emphasis on continuous integration means that performance regressions are detected early. However, some XP practices assume a high level of software abstraction that may be less applicable to low-level embedded code running on resource-constrained hardware like Arduino.

3. Maintainability

Maintainability refers to the effectiveness and efficiency with which a system can be modified to correct defects, improve performance or adapt to changes in requirements (Sheppard, 2025). For the fly-by-wire system, maintainability is important because the Arduino software will likely require adjustments throughout the projects as sensor tuning is refined, control algorithm parameters are tuned, and hardware components are changed or upgraded. A codebase that is poorly structured difficult to understand will make these adjustments slow and error prone.

How each Agile methodology supports Maintainability:

- Scrum: Scrum supports maintainability through its retrospective ceremony. Watt (2014) explains that Scrum involves reviewing what was done at the end of each sprint, which creates a structured opportunity for the team to reflect on technical debt, identity code quality problems and plan corrective work in the next sprint
- Kanban: Kanban supports maintainability by allowing maintenance and refactoring tasks to be added to the workflow at any time. When defect or a structural problem is identified during hardware testing, it can be added to the board and prioritised without waiting for a sprint boundary, which is useful in a project where physical testing often uncovers unexpected issues.
- Extreme Programming: XP supports maintainability mostly through its emphasis on improving engineering practices. Watt (2014) notes that one of the Scrum Master's responsibilities and a shared concern across Agile methodologies is to improve engineering practices and tools. XP formalises this through practices such as refactoring, collective code ownership and consistent coding standards which keep the embedded codebase clean and modular.

4. Functional Suitability

   Functional Suitability assesses whether a system provides the functions that meet the stated needs of its users under specified conditions (Sheppard, 2025). For the fly-by-wire Arduino system, functional suitability means the software must correctly read the gyroscope and accelerometer data, calculate the required fin deflection and send the

appropriate PVM signal to the actuator. Errors in any of these functions, even minor ones, could compromise the aerodynamic stability of the aircraft model during testing.

How each Agile methodology supports Functional Suitability:

- Scrum: Scrum supports functional suitability through its product backlog. Watt (2014) describes the backlog as a collection of stories and tasks, with the product owner and development team negotiating the scope of each sprint. This ensures that all required functions are explicitly identified, prioritised and tracked to completion.
- Kanban: Kanban makes all functional tasks visible on a continuous flow board. This helps the team see which functions have been completed and which remain outstanding, but Kanban does not prescribe a formal process for capturing or prioritising functional requirements, which may be a limitation for a project with complex and interdependent system functions.
- Extreme Programming (XP): XP supports functional suitability through the acceptance testing and close engagement with project stakeholders. Watt (2014) identifies that the product owner as a stakeholder who must be intimately involved in sprint planning and demonstration and a similar role exists in XP. By writing acceptance tests before development begins and the team defines precisely what each function must do, reducing the risk of functional errors in the delivered system.

6. Comparative Analysis of Methodologies

6.1 Similarities Between the Methodologies

All three methodologies are rooted in Agile values and share commitment to iterative, incremental delivery. Watt (2014) describes Scrum as based on the recognition that it is not realistic to plan an entire project upfront and that teams should instead build empirical data and replan as they iterate. This principle is shared by Kanban and XP and all three encourage the team to respond to what they learn during development rather than following a rigid, fixed plan. This is especially valuable for a hardware integrated product like this one, where each physical test of the aircraft model may reveal new requirements or expose incorrect assumptions about system behaviour.

All three methodologies also support transparency and communication. Watt (2014) notes that Scrum uses a short daily meeting to check what was done yesterday, what is planned for the next day and whether anything is impeding the team. Kanban achieves similar visibility through its board and XP through pair programming and shared code ownership. Each approach ensures that team members remain aware of progress and can respond quickly to emerging problems.

6.2 Key Differences and Trade-offs

While all three methodologies are Agile, they differ meaningfully in structure, planning rigour and technical guidance. These differences have implications for this project.

Scrum provides the most structured of the three frameworks. Watt (2014) describes its core roles, the Product Owner, the Scrum Master and the Development Team, each with defined responsibilities. The product owner manages and prioritises the backlog while the Scrum Master removes obstacles and facilitates team productivity and the Development Team plans, builds and demonstrates work each sprint. This structure suits this project team because it establishes clear accountability and provides natural points, sprint reviews and planning sessions, at which the team can assess progress and adjust plans.

Watt (2014) also notes that Scrum uses metric such as burn-down charts and velocity to support planning and progress tracking. These tools give the team a concrete view of how much work remains and how much the team can realistically complete in a given sprint, which is important when managing a project with a fixed academic deadline.

Kanban, in contrast, operates as a continuous flow system with no fixed iterations. Work items move through stages on a board, and the output rate is managed through Work-In-Progress (WIP) limits. This makes Kanban highly flexible, but it lacks Scrum's planning and role structure. For a project that involves designing and integrating a new embedded system from scratch, the absence of sprint planning and backlog management makes it harder to coordinate interdependent development tasks.

XP is the most technically prescriptive of the three. Watt (2014) groups XP with Scrum as part of the broader category Agile project management methodologies and notes that improving engineering practices and tools is a central concern of Agile teams. XP formalises this through specific practices including test-driven development, pair programming, refactoring and continuous integration. These practices directly improve code quality and

reliability, but they also require a higher level of engineering discipline and are more demanding to apply to hardware-facing embedded code than to conventional application software.

6.3 Comparison Table

	Scrum	Kanban	XP
Reliability	Good, iterative testing each sprint improves system stability over time (	Moderate, WIP limits reduce instability risk but not structured	Very good, TDD and CI directly and continuously support reliability.
Performance Efficiency	Good, performance targets included in sprint goals (Watt, et al., 2014)	Good, performance fixes introduced immediately without sprint boundaries	Moderate, CI detects regressions, but embedded optimisation may need manual effort
Maintainability	Moderate, retrospectives enable identification of technical debt	Good, maintenance tasks easily added to board at any time	Very good, refactoring and coding standards are core practices
Functional Sustainability	Good — backlog ensures all functions are defined, prioritised, and tracked (Watt, et al., 2014)	Moderate — visual board tracks tasks but no formal requirements process	Good, acceptance tests define and verify each function precisely

7. Selection and Justification of Methodologies

Based on the comparative analysis diligently done in Section 6, Scrum is the agile methodology that makes the most sense for the Fly-by-Wire (FBW) Arduino System project. Scrum, Kanban, and Extreme Programming (XP) all share the core Agile principles like adaptability, iterative development, and team collaboration the differences in planning, and structure makes Scrum the most suitable for this project ahead of the other methodologies.

The main reason Scrum was chosen was because it provides clearly defined roles to ensure clear accountability and makes for organised collaboration. The structure and planning work well to help overcome potential challenges like integrating the hardware and software components. In a project such as this with many moving parts like sensors, actuators, control algorithms, etc... it can be easy to make a mess of something hence the structure, planning, and accountability are so important.

Scrum makes use of time-boxed sprints to allow the team to breakdown complex tasks into more manageable increments which helps with the development of the overall system. This enables systematic development and testing of individual components which reduces possible integration risks. Sprint Reviews and Retrospective analysis mean that there are efforts being made on evaluations and improvements. This ensures that problems are identified and addressed early which helps manage risk as well. In Section 6 tools such as velocity, and burndown charts are discussed. These tools give the team a clear understanding of progress made, and the amount work still to be done. In projects with strict deadlines, and specific deliverables Scrum allows for realistic planning to ensure timely achievement of milestones and deadlines due to its inclusion of these tools in tracking progress.

Methodologies like Kanban and XP each have their strengths and have great offerings, but in the context of this project, they have some limitations. Kanban provides a high level of flexibility through its continuous workflow; however, it lacks the structured planning, defined roles and formal review attributes that Scrum is made up of. This makes Kanban less suitable for managing the technical and interdependent tasks and components involved in this project, especially when looking at coordinating the integration and delivering on the scheduled deliverables. XP offers strong support for reliability and maintainability through test-driven development, refactoring, and continuous integration. Only flop is that it is heavily focussed on software engineering principles and requires a high level of technical discipline which makes it less suitable for systems where hardware integration is also a valuable component in the final project deliverable.

To conclude, Scrum provides the most balanced approach by combining structure, progress tracking, iterative development, and risk management. It is the most appropriate methodology to successfully deliver the Fly-by-Wire Arduino system within the constraints, and scope.

8. Agile Tools and Technologies

To support the implementation of a Fly-by-Wire Arduino System, the project will require a vigorous environment which needs to be capable of handling both the embedded software development as well as the physical hardware assembly milestones.

8.1 Evaluation Criteria

To find the most appropriate tool these criteria were used:

- **Hardware-Software Traceability:** The ability to link the stabilization algorithm code (e.g. C++/ C) to the physical hardware to test the results.
- **Integration Capability:** Compatibility with version control to manage the PID control logic as well as the IOT dashboard telemetry data.
- **Phase Management:** The effectiveness of tracking the specific project methodology, from the 3D printing of the model to the final system evaluation.
- **Collaboration:** The ease of which real-time progress on the sensors and actuators integration can be shared with the supervisors.

8.2 Tool Comparison

- **Project Fit:** Jira is highly suited for the technical software development and AI model implementation, whereas Trello is best for the high-level visual tracking of the physical prototype phases e.g. the 3D printing phase.
- **Complexity:** Jira is highly recommended here because it allows for detailed issue tracking regarding the sensor calibration and the logic bugs, where Trello is not recommended because it uses a simple Kanban board to move task from “In-Scope” to “Done”.
- **Reporting:** Jira is robust meaning that it includes automated Velocity and Burndown chats which are essential for engineering reports. Trello is very basic and is limited to simple activity logs without the advanced Agile metrics.

8.3 Recommendation and Justification

- **Recommendation:** Jira is highly recommended for this project.
- **Justification:** Because the implementation of AI algorithms to detect that a pilot is incapacitated is very complex and we need real-time data processing, Jira is perfect because it has advanced issue-tracking capabilities which are vital. It allows us to document the specific technical failures during the “Testing & Calibration” phase, which ensures that all six of the project objectives are systematically met and verified.

9. Agile Metrics and Progress Tracking

The following metrics will be used to monitor the development of the FBW prototype and support the data-driven decision making throughout the project's lifecycle.

9.1 Selected Metrics and Justification

- **Velocity:** This will track the number of technical tasks which are completed each sprint.
    - **Justification:** It helps us to estimate if the project is on track to be able to deliver the "Final project report" by the deadline.
- **Breakdown Charts:** This is to provide a day-to-day visual of the work remaining within the current development phase, such as the "Prototype Development" phase.
    - **Justification:** It will allow us to identify immediately if there are hardware delays, such as 3D printing failures, which can impact the software integration timeline.
- **Cycle Time:** This will allow us to measure the time taken from coding a specific servo actuation logic to the successful calibration and testing.
    - **Justification:** A high cycle time in the "Software Development" phase would signal that the PID control logic needs to be simplified or that we need to seek technical support.

9.2 Monitoring and Decision-making

These metrics will be reviewed during every weekly meeting to ensure that the project remains within the defied scope. For instance, if the Velocity drops significantly, we may need to decide whether to prioritize Objective 2 over the optional features like the manual override joystick to ensure that the core fly-by-wire system is functional for the final demonstration.

10. Risk and Adaptability Considerations

As with any choice of project management ideology, there are considerations that need to be made regarding the potential risks surrounding the chosen approach, and the adaptability it can provide between differing workflows. This section will explore those considerations following our choice of the Scrum Agile methodology.

Risks:

- Sprint delays caused by hardware dependencies: Hardware components may arrive late or be faulty upon arrival. This can prevent software tasks from being completed within a sprint.
- Difficulty estimating technical tasks: Embedded systems like these often involve unpredictable debugging, making it difficult to accurately estimate how much work could be completed during a sprint.
- Integration complexity: Software and hardware developed separately may not function as intended when combined, which could create unexpected issues during sprint reviews.
- Increased documentation pressure: Academic projects require formal documentation in addition to development work. Scrum naturally doesn't require extensive documentation, so this effort will have to be deliberate and time-consuming.
- Team coordination challenges: Scrum relies on regular and thorough communication. Poor communication between members, therefore, can result in incomplete, duplicated, missing or incorrect work.

Adaptability:

- Sprint reviews: At the end of each sprint, the team can evaluate progress and modify priorities based the results of testing and technical issues that were discovered.
- Backlog prioritisation strategies: Arising technical problems or supervisor feedback can be added to the product backlog and prioritised for future sprints.
- Incremental development: The system can be built in smaller stages and modules, allowing faults to be identified early before they affect the entire prototype.
- Retrospective analyses: Regular reflection allows the team to improve communication and adjust development practices throughout the project.
- Parallel development support: Scrum can accommodate separate hardware and software tasks while still integrating them into singular sprint goals.

11. Conclusion and Recommendations

Looking at all the information we just covered, we can come to a reasonable conclusion. This report evaluated three Agile methodologies, namely Scrum, Kanban, and Extreme Programming (XP), in the context of developing a scaled fly-by-wire Arduino system. Each methodology showed alignment with core Agile principles such as iterative development, adaptability, and continuous improvement, but their suitability differed when considering project-specific factors like hardware-software integration, complexity management, risk mitigation, and academic delivery requirements.

From the comparison, it was found that Kanban offers flexibility and a continuous workflow, however it does not provide the structured planning and clear role definition needed to manage a technically complex and interdependent system. XP includes strong engineering practices that improve reliability and code maintainability, but its focus is heavily software-oriented, which makes it less ideal for a project that also involves significant hardware development.

Scrum was identified as the most suitable methodology for this project. Its structured framework, which includes defined roles, time-boxed sprints, and regular review mechanisms, supports coordination between hardware and software activities. It allows the team to break large tasks into smaller, manageable increments, helps detect integration issues early, and keeps stakeholders involved through sprint reviews. Tools like velocity tracking and burndown charts also make it easier to monitor progress and stay on track with academic deadlines.

Based on these findings, it is recommended that the project adopt Scrum as the primary methodology. To get the most out of it, the team should do the following:

Roles such as Scrum Master, Product Owner, and Development Team should be clearly defined to ensure accountability and collaboration.

A well-prioritised product backlog should be maintained that covers both technical tasks and academic deliverables.

Short and consistent sprint cycles should be used to allow for regular hardware-software integration testing.

Sprint reviews and retrospectives should be held consistently to catch problems early and keep improving the process.

Where relevant, certain XP practices like continuous integration and refactoring can be used alongside Scrum to improve code quality.

By following this approach, the team will be in a better position to handle the complexity of the project, reduce risks, and deliver a working fly-by-wire prototype within the required timeframe and academic constraints.

References

Backblaze, 2015. *Applying Agile Methodology to Hardware Development.* [Online]
Available at: https://www.backblaze.com/blog

BMC Blogs, 2025. *Scrum Framework and Agile Basics.* [Online]
Available at: https://www.bmc.com/blogs

Cordeiro, L., Barreto, R. & Barcelos, R., 2007. TXM: an agile HW/SW development methodology for building medical devices. *ACM SIGSOFT Software Engineering Notes,* pp. 1-4.

Sheppard, E., 2025. *ISO/IEC 25010:2023 Systems and software engineering Systems and software Quality Requirements and Evaluation (SQuaRE) — Product quality model characteristics, sub-characteristics, definitions,and their proposed application to electric vehicle supply equip,* s.l.: Zenodo.

TOJDEL, 2017. Project Management and Agile Frameworks. *The Online Journal of Distance Education and e-Learning.*

Wang, X., Smith, J. & Doe, J., 2018. Agile Software Development in Safety-Critical Environments. *Journal of Systems and Software.*

Watt, A., B. & B.C, V., 2014. *Project Management.* s.l.:s.n.